In the above example, for the code in *file1.c*, *sizeof()* can be applied to find the size of the array, as it is completely defined in *file1.c*. But, in *file2.c*, the *sizeof()* operator will not work since the dimensions of the array are missing. Without this information, the compiler has no knowledge of how many elements are in the array and cannot calculate the *sizeof* of the array.

How *sizeof()* is different from a function call

Let us consider the following code to understand how *sizeof()* is different from a function call:

```
#include <stdio.h>
#include <stddef.h>

int main()
{
        int x = 5;

  printf("%lu:%lu:%lu\n", (long unsigned )sizeof(int),
   (long unsigned )sizeof x, (long     unsigned )sizeof 5);

        return 0;
}
```

In the above example, *sizeof()* will work even if the braces are not present across operands, whereas in functions, braces are a must. So, here are three reasons why *sizeof* is not a function:

1. It can be applied for any type of operand.
2. It can also be used, when type is an operand.
3. No brackets needed across operands

You can implement your own *SIZEOF()* macro, which should work like a *sizeof()* operator.

According to the C99 standards, the *sizeof()* operator yields the size (in integer bytes) of its operand, which may be an expression or the parenthesised name of a type. If the type of the operand is a variable length array type, the operand is evaluated at runtime; otherwise, the operand is not evaluated and the result is an integer constant, during the compile time itself. END

By: Satyanarayana Sampangi

The author is a member of the embedded software team at Emertxe Information Technology (P) Ltd *(http://www.emertxe.com)*. His areas of interest are embedded C programming combined with data structures and microcontrollers. He can be reached at *satya@emertxe.com*